

COMPILER DESIGN LABORATORY

PROJECT INSTRUCTION MANUAL -- PHASE 1 SUBMISSION

ZENITH COMPILER

A Strongly-Typed Procedural Language Compiler with AST Visualization,
Scoped Symbol Table Management, and Intermediate Code Generation

PHASE 1 DELIVERABLES SPECIFICATION (MANUAL SECTION 8.3)

This project submission strictly fulfills the deliverables mandated in Section 8.3 of the Compiler Design Laboratory Project Instruction Manual: Project Title, Abstract, Problem Statement, Motivation, Objectives, Scope, Background Study, Compiler Design Concepts Involved, Proposed Methodology, System Architecture, Technology Stack, and Initial Working Prototype.

PROJECT METADATA & SUBMISSION DETAILS

- Project Type: Individual Project Submission (Strictly Independent Implementation)
- Course Title: Compiler Design Laboratory
- Review Milestone: Review 1: Project Proposal and System Design
- Evaluation Phases: Phase 1: Design & Prototype | Phase 2: Core Implementation | Phase 3: Final System
- Project Repository: /zenith-compiler (Source Code, Test Harness, and Documentation)

TABLE OF CONTENTS

Section 1	Executive Abstract & Technical Keywords	Page 2
Section 2	Project Title, Motivation & Problem Statement	Page 3
Section 3	Project Objectives & Scope Boundaries	Page 4
Section 4	Background Study & Literature Survey	Page 5
Section 5	Compiler Design Concepts Involved	Page 6
Section 6	Proposed Methodology & Lifecycle Model	Page 7
Section 7	System Architecture & Component Interaction	Page 8
Section 8	Technology Stack & Language Specification	Page 9
Section 9	Initial Prototype Verification & Execution Evidence	Page 10
Section 10	Symbol Table Verification, Roadmap & References	Page 11

1. EXECUTIVE ABSTRACT

This report details the formal design, architectural specification, and initial prototype implementation of Zenith, a strongly-typed procedural programming language and its accompanying multi-pass compiler. In contemporary software engineering, developers encounter a persistent trade-off: dynamically typed scripting languages postpone type and scoping validation to runtime, causing catastrophic unhandled exceptions in production; while monolithic industrial compilers operate as opaque black boxes where internal syntax trees and intermediate representation lowering cannot be inspected or verified for academic analysis.

The Zenith compiler project resolves this challenge by formulating a transparent, verifiable multi-pass language processing pipeline grounded in classical compiler theory. The compiler front-end implements a flexible interactive execution engine capable of receiving arbitrary user input, tokenizing lexemes with coordinate tracking, constructing an Abstract Syntax Tree via recursive descent parsing, and resolving nested lexical scopes through an environment tree symbol table.

In strict compliance with Section 8.3 of the lab manual, Phase 1 establishes the problem definition, formal EBNF grammar, modular system architecture, and an initial working prototype verified on real computational programs. All front-end components have been engineered with zero third-party dependencies.

- Technical Keywords: Compiler Design, Lexical Scanning, LL(1) Parsing, Recursive Descent, Abstract Syntax Tree, Scoped Symbol Table, Static Typing, Interactive REPL.

2. PROJECT TITLE, MOTIVATION & PROBLEM STATEMENT

2.1 Project Title

Formal Title: Zenith: A Strongly-Typed Procedural Language Compiler with AST Visualization, Scoped Symbol Table Management, and Intermediate Three-Address Code Generation.

Short Title for Submission: Design and Implementation of the Zenith Language Compiler.

2.2 Motivation

The motivation for constructing the Zenith compiler is centered on addressing three critical educational and engineering challenges:

- 1. Static Safety vs. Scripting Usability: While scripting languages offer convenient syntax, they lack compile-time safety guarantees. Zenith combines clean procedural syntax with compile-time type checking and immutability tracking.
- 2. Arbitrary User Problem Input: Rather than restricting execution to static hardcoded files, the Zenith compiler front-end incorporates an interactive input engine. Users can input any computational problem or algorithmic logic dynamically, and the compiler immediately scans, parses, and validates the input.
- 3. Pedagogical Transparency: Industrial compilers (GCC, Clang) obscure intermediate compiler data structures. Zenith explicitly exposes token tables, Abstract Syntax Trees, and scoped symbol tables directly in terminal output.

2.3 Formal Problem Statement

To formulate, design, implement, and validate an end-to-end multi-pass compiler system for the Zenith language that satisfies:

- Scanning Requirement: Convert raw user source text into categorized tokens while maintaining line and column coordinate metadata for precise diagnostic feedback.
- Parsing Requirement: Parse the token stream using a deterministic LL(1) recursive descent parser adhering to an unambiguous context-free grammar with operator precedence climbing.
- Translation Requirement: Synthesize a strongly-typed Abstract Syntax Tree (AST) eliminating syntactic punctuation while preserving operator binding precedence.
- Scoping Requirement: Manage nested lexical environments via an environment tree symbol table, enforcing variable declarations and immutability constraints.

3. PROJECT OBJECTIVES AND SCOPE BOUNDARIES

3.1 Primary Project Objectives

- 1. Formal Grammar Specification: Formulate an unambiguous Context-Free Grammar in Extended Backus-Naur Form (EBNF) supporting procedural programming constructs.
- 2. Hand-Crafted Scanner: Develop a high-performance lexical analyzer recognizing keywords, identifiers, literals, and multi-character operators with coordinate tracking.
- 3. Recursive Descent Parser: Implement a top-down parser with operator precedence climbing and panic-mode error recovery at statement boundaries.
- 4. AST Construction: Formulate an object-oriented Abstract Syntax Tree hierarchy with visual ASCII tree formatting for inspection.
- 5. Scoped Symbol Table: Implement an environment tree data structure supporting parent-pointer chaining and duplicate declaration prevention.
- 6. User Input Engine: Deliver an interactive command-line interface and REPL allowing arbitrary user-supplied programs to be tested dynamically.

3.2 Scope Boundaries

To maintain rigorous implementation depth and ensure full project feasibility across all three phases, project boundaries are defined:

IN-SCOPE FUNCTIONALITY

- Primitive Data Types: int, float, bool, string, and void. - Variable Bindings: Mutable ('let') and immutable ('const') declarations. - Comprehensive Operators: Arithmetic (+, -, *, /, %), Relational (==, !=, <, <=, >, >=), Logical (&&, ||, !). - Structured Control Flow: Multi-way branching (if-elif-else), while loops, and initialized for loops. - Functional Abstractions: Function declarations with typed formal parameters and explicit return types. - Intermediate Representation: Linearized Three-Address Code (TAC) quadruples with temporary registers. - Code Optimization: Machine-independent constant folding, algebraic simplification, and dead code elimination. - Execution Engine: Stack-based

OUT-OF-SCOPE BOUNDARIES (DESIGN NON-GOALS)

- Dynamic Garbage Collection: Memory follows structured lexical activation records; complex heap GC is excluded. - Polymorphic OOP Hierarchies: Virtual method tables and multi-inheritance are excluded to preserve compiler pipeline clarity. - Hardware Assembly Emission: Direct emission of x86/ARM machine code is replaced with Three-Address Code and bytecode execution to eliminate machine-specific linking quirks.

4. BACKGROUND STUDY & LITERATURE SURVEY

4.1 Theoretical Foundations of Language Processing

Compiler construction is an established discipline of computer science. As detailed in canonical literature such as *Compilers: Principles, Techniques, and Tools* (Aho, Lam, Sethi, and Ullman), a compiler translates high-level program representations into equivalent executable representations through discrete analytical and synthetic phases.

4.2 Lexical Analysis & Finite Automata

Lexical analysis (scanning) groups input characters into classified tokens defined via Regular Expressions (RE). By Thompson's Construction, regular expressions translate into Non-deterministic Finite Automata (NFA), which convert via subset construction into Deterministic Finite Automata (DFA). While tools like Lex and Flex generate DFA tables, a hand-crafted scanner provides superior flexibility for coordinate tracking (line and column) and user-friendly error reporting.

4.3 Parsing Techniques: Top-Down vs. Bottom-Up

Syntax analysis determines if a token sequence conforms to a Context-Free Grammar (CFG). Two primary parsing paradigms exist:

- Bottom-Up Parsers (LR, LALR): Used in tools like Yacc and Bison. They operate via shift-reduce actions over a state stack. Although expressive, LALR parsers suffer from obscure shift-reduce conflicts and difficult-to-customize error diagnostics.
- Top-Down Parsers (LL(k), Recursive Descent): Construct the parse tree from root to leaves. Hand-written recursive descent parsers directly mirror grammar productions in programming code. Combined with precedence climbing, they provide unmatched diagnostic clarity, zero build dependencies, and easy verification.

4.4 Abstract Syntax Trees (AST) vs. Concrete Parse Trees

A Concrete Parse Tree (CST) contains all punctuation tokens and intermediate non-terminal expansions. An Abstract Syntax Tree (AST) condenses the parse tree into an operator-operand hierarchy, discarding syntactic noise while preserving semantic relationships.

4.5 Intermediate Representations (Three-Address Code)

Intermediate Representations decouple the front-end language syntax from the back-end execution architecture. Three-Address Code (TAC) linearizes nested expression trees into instructions with at most three addresses, simplifying optimization and execution.

5. COMPILER DESIGN CONCEPTS INVOLVED

The Zenith compiler directly implements and demonstrates key theoretical concepts from the Compiler Design curriculum:

5.1 Regular Expressions & DFA Tokenization

The Lexical Analyzer recognizes 34 distinct token types using regular expressions. It maintains source coordinates (line and column), processes string escape characters, strips single-line (`//`) and multi-line (`/* */`) comments, and uses lookahead matching to resolve multi-character operator ambiguities (`=` vs `==`, `<` vs `<=`, `-` vs `->`).

5.2 Context-Free Grammars & EBNF Specification

The language syntax is governed by a formal 4-tuple $G = (V, \Sigma, R, S)$, where V is non-terminals, Σ is terminal tokens, R is production rules, and S is the start symbol. The grammar is structured in Extended Backus-Naur Form (EBNF) to eliminate left recursion.

5.3 Operator Precedence Climbing

Expressions are parsed using operator precedence climbing across seven priority tiers, guaranteeing that higher-precedence operations (multiplication, division) bind tighter than lower-precedence operations (addition, relational comparisons, and logical operations).

5.4 Hierarchical Scoped Symbol Tables

Lexical scoping is modeled as an environment tree with parent pointers. Each scope records identifier name, type, category (variable, constant, function), declaration line, and scope level. Duplicate declarations in the same scope are caught in $O(1)$ time, while lookups traverse up to the global scope.

5.5 Panic-Mode Error Recovery & Synchronization

The parser implements panic-mode recovery: upon encountering a syntax error, it logs coordinate-annotated diagnostics and discards tokens until a synchronizing statement boundary (semicolon or keyword like `'let'`, `'if'`, `'while'`, `'fn'`) is reached, allowing multiple syntax errors to be reported in a single run.

6. PROPOSED METHODOLOGY & DEVELOPMENT LIFECYCLE

6.1 Multi-Pass Compiler Engineering Lifecycle

The project follows a phased, multi-pass engineering methodology where each translation phase produces a verified intermediate data structure:

- Stage 1: Lexical Specification: Formal definition of token types, regex patterns, keyword mappings, and coordinate tracking.
- Stage 2: Grammar Formulation: Formulation of unambiguous EBNF grammar rules and operator precedence hierarchy.
- Stage 3: Front-End Construction: Implementation of Lexer, Recursive Descent Parser, and object-oriented AST node hierarchy.
- Stage 4: Scope & Symbol Management: Implementation of hierarchical environment tree and scope resolution builder.
- Stage 5: Test-Driven Verification: Validation against diverse test suites covering math expressions, control flow, functions, and error recovery.

6.2 Phase-wise Implementation Breakdown

Implementation Phase	Core Functional Modules	Review Target & Deliverable Focus
Phase 1 (Completed)	Grammar, Lexer, Parser, AST Node Definition	Visualize, Symbol Table, System Design & Prototype
Phase 2 (Upcoming)	Type Checker, Semantic Analyzer, IR Code Generation	Review IR Code, Address Calculation & Intermediate Representation
Phase 3 (Upcoming)	Constant Folding Optimizer, Variable Allocation, Final System	Review Final System, Optimization & Testing

6.3 Verification & Quality Assurance Strategy

The verification strategy utilizes test suites and interactive user input to exercise every language feature:

- Basic Math Suite: Validates integer/float literals, operator precedence climbing, and boolean relational evaluations.
- Control Flow Suite: Validates branch parsing in multi-branch if-elif-else statements, while loop condition evaluation, and for loop initialization.
- Functions Suite: Validates function declaration parsing, typed parameter lists, return statements, and local scope creation.
- User Interactive Engine: Accepts arbitrary user-supplied programs at runtime, verifying scanning, parsing, and scoping dynamically.

7. SYSTEM ARCHITECTURE & COMPONENT DESIGN

7.1 End-to-End Architectural Pipeline

The Zenith compiler is organized into a modular multi-pass pipeline where each component operates on explicit data structures:

ZENITH COMPILER ARCHITECTURAL DATA FLOW

```
Source Code (.zen or User Input) ---> [Lexical Analyzer (lexer.py)] ---> Token
Stream | v Token Stream -----> [Recursive Descent Parser
(parser.py)] ---> Abstract Syntax Tree (AST) | v Abstract Syntax Tree
-----> [Symbol Table Builder (symbol_table.py)] ---> Scoped Environment
Tree | v AST & Symbol Table -----> [Terminal Visualizer & CLI
(printer.py / main.py)] ---> Structured Diagnostics
```

7.2 Component Role Descriptions

- Lexical Analyzer (src/lexer.py): Scans character buffers, identifies lexemes, checks keyword dictionaries, tracks line:column coordinates, and logs lexical errors.
- Token Definitions (src/tokens.py): Defines the TokenType enumeration (34 distinct token categories) and the immutable Token dataclass.
- AST Node Hierarchy (src/ast_nodes.py): Object-oriented dataclass hierarchy representing statements (VarDecl, IfStmt, WhileStmt, FnDecl) and expressions (BinaryExpr, LiteralExpr, CallExpr).
- Recursive Descent Parser (src/parser.py): Enforces EBNF grammar productions, executes precedence climbing, handles lookahead, and implements panic-mode recovery.
- Scoped Symbol Table (src/symbol_table.py): Manages hierarchical lexical scopes (global, function, block), validates identifier declarations, and enforces immutability.
- AST Visualizer & Printer (src/prINTER.py): Renders ASCII tree representations of the AST and tabular summaries of tokens and symbol tables.
- CLI Driver & REPL (src/main.py): Provides the command-line interface, argument flags (--tokens, --ast, --symtab, --all), and an interactive shell.

8. TECHNOLOGY STACK & LANGUAGE SPECIFICATION

8.1 Technology Selection Justification

The compiler is implemented in modern Python 3 (standard library only). This technology choice provides significant advantages for laboratory evaluation:

- Zero External Toolchain Dependencies: Eliminates build and linking errors associated with C/C++ Flex/Bison configurations across different operating systems.
- High Diagnostic Transparency: Python's dataclasses, pattern matching, and recursion support clean, readable implementations of tree traversal algorithms.
- Portability & Demonstration: Executes out-of-the-box on macOS, Linux, and Windows without binary recompilation.

8.2 Formal Context-Free Grammar (EBNF Summary)

The Zenith grammar is structured in Extended Backus-Naur Form (EBNF) to guarantee deterministic LL(1) derivation:

- Program Level: `program ::= { statement }`
- Statement Level: `statement ::= var_decl | assignment | if_stmt | while_stmt | for_stmt | fn_decl | return_stmt | print_stmt | block`
- Variable Declarations: `var_decl ::= ('let' | 'const') IDENTIFIER ':' type ['=' expression] ';' ;`
- Conditionals: `if_stmt ::= 'if' '(' expression ')' block { 'elif' '(' expression ')' block } ['else' block]`
- Loops: `while_stmt ::= 'while' '(' expression ')' block | for_stmt ::= 'for' '(' [var_decl] [expression] ';' [assignment] ')' block`
- Functions: `fn_decl ::= 'fn' IDENTIFIER '(' [param_list] ')' ['->' type] block`

8.3 Operator Precedence and Associativity Table

Precedence Level	Operator Class	Associativity & Binding Power
Level 1 (Lowest)	Logical OR ()	Left-to-Right evaluation
Level 2	Logical AND (&&)	Left-to-Right evaluation
Level 3	Equality (==, !=)	Left-to-Right evaluation
Level 4	Relational (<, <=, >, >=)	Left-to-Right evaluation
Level 5	Additive (+, -)	Left-to-Right evaluation
Level 6	Multiplicative (*, /, %)	Left-to-Right evaluation
Level 7 (Highest)	Unary (-, !)	Right-to-Left prefix binding

9. INITIAL PROTOTYPE VERIFICATION & EXECUTION EVIDENCE

The Zenith compiler front-end incorporates an interactive user input checker (run_interactive.py), allowing any arbitrary computational problem or algorithmic snippet to be taken as input directly from the user at runtime. The compiler scans, parses, and validates the program, displaying the Token Stream Table, the Abstract Syntax Tree, and the Scoped Symbol Table.

The following real execution evidence demonstrates the system analyzing a circle area computation problem taken as input from the user:

=====				
COMPILATION ANALYSIS RESULT				
=====				
[STEP 1: LEXICAL ANALYSIS (SCANNER)]				
Tokens Table:				
Line	Col	Token Type	Lexeme	Value
1	1	CONST	const	-
1	7	IDENTIFIER	PI	-
1	9	COLON	:	-
1	11	IDENTIFIER	float	-
1	17	ASSIGN	=	-
1	19	IDENTIFIER	3.14159	3.14159
1	26	SEMICOLON	;	-
2	1	LET	let	-
2	5	IDENTIFIER	r	-
2	6	COLON	:	-
2	8	IDENTIFIER	float	-
2	14	ASSIGN	=	-
2	16	IDENTIFIER	10.0	10.0
2	20	SEMICOLON	;	-
3	1	LET	let	-
3	5	IDENTIFIER	circle_area	-
3	16	COLON	:	-
3	18	IDENTIFIER	float	-
3	24	ASSIGN	=	-
3	26	IDENTIFIER	PI	-
3	29	STAR	*	-
3	31	IDENTIFIER	r	-
3	33	STAR	*	-
3	35	IDENTIFIER	r	-
3	36	SEMICOLON	;	-
4	1	PRINT	print	-
4	6	L_PAREN	(	-
4	7	STRING_LITERAL	"Circle area is:"	Circle area is:
4	24	COMMA	,	-
4	26	IDENTIFIER	circle_area	-
4	37	R_PAREN	)	-
4	38	SEMICOLON	;	-
4	39	EOF		-

[STEP 2: SYNTAX ANALYSIS & ABSTRACT SYNTAX TREE]				
Abstract Syntax Tree (AST):				
Program [L1:C1]				
├─ VarDecl: const PI: float [L1:C1]				
│ └─ Literal: 3.14159 (float) [L1:C19]				
├─ VarDecl: let r: float [L2:C1]				
│ └─ Literal: 10.0 (float) [L2:C16]				
├─ VarDecl: let circle_area: float [L3:C1]				
│ └─ BinaryExpr: (*) [L3:C30]				
│ └─ BinaryExpr: (*) [L3:C29]				
│ └─ Identifier: PI [L3:C26]				
│ └─ Identifier: r [L3:C31]				
│ └─ Identifier: r [L3:C35]				
└─ PrintStmt [L4:C1]				
└─ Literal: 'Circle area is:' (string) [L4:C7]				
└─ Identifier: circle_area [L4:C26]				

[STEP 3: SCOPED SYMBOL TABLE]				
=====				

Figure 1: User Interactive Execution Output -- Lexical Analysis & Abstract Syntax Tree

Shows the Lexer token table (coordinates, lexemes, types) and the Parser AST tree for user-input circle area calculation.

10. SYMBOL TABLE VERIFICATION, ROADMAP & REFERENCES

[STEP 3: SCOPED SYMBOL TABLE]				
=====				
ZENITH SCOPED SYMBOL TABLE				
=====				
[Scope: 'global'] (Level: 0, Parent: 'None')				
Name	Type	Category	Defined At	Details
PI	float	const	L1:C1	scope_lvl=0
r	float	let (var)	L2:C1	scope_lvl=0
circle_area	float	let (var)	L3:C1	scope_lvl=0

 Verification Successful: All tokens valid, grammar matched, and symbols scoped!

Figure 2: User Interactive Execution Output -- Scoped Symbol Table & Verification

Shows the global scope symbol table with identifier types, categories (const vs let), coordinates, and validation status.

10.1 Implementation Roadmap for Phase 2 and Phase 3

- Phase 2 Target (Core Implementation): Implement semantic type checking, return type validation, immutability checking for const variables, and generation of linearized Three-Address Code (TAC) quadruples.
- Phase 3 Target (Final Implementation): Implement machine-independent TAC optimizations (constant folding, dead code elimination) and construct a lightweight stack-based virtual machine runtime.

10.2 Academic References

- 1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006): Compilers: Principles, Techniques, and Tools (2nd Edition). Addison-Wesley.
- 2. Cooper, K. D., & Torczon, L. (2011): Engineering a Compiler (2nd Edition). Morgan Kaufmann.
- 3. Loudon, K. C. (1997): Compiler Construction: Principles and Practice. PWS Publishing.
- 4. Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J., & Langendoen, K. (2012): Modern Compiler Design. Springer.